
Main script

```
c=input('enter a natural nubmer: ');
% the list of factors, starting from an empty
pf=[];
if c==1
    disp('empty prodct')
    % case c 1 one, unique case
end
global squarenumbers
squarenumbers=(0:floor(1/2*(c/3+3))).^2;
% the list of squarenumbers
while mod(c,2)==0
    % if c is odd, we just divide by 2
    pf(end+1)=2;
    c=c/2;
end
if c~=1
    pf(end+1)=c;
end
while true
    if c==1
        % we change c value so we have to check if c is not 1 every time
        break
    % if c=1, break this loop
end
for i=1:length(pf)
    % apply function to each factors
    check=0;
    % if we applied the function (1) or not (0)

    if isprime(pf(i))==0
        % apply function if the number in factor set is not prime
        prime_factor(pf(i));
        % recall the function
        pf(i)=[];
        % erace the number we just factored from
        pf(end+1)=ans(1);
        % here, ans(1)=a+b
        pf(end+1)=ans(2);
        % here, ans(2)=a-b
        % add two factors in the list
        check=1;
        % tell it that we did prime factor process
        break
    end
end
if check==0
    % this means that we didn't apply prime factorization to non of the
    % element
    break
end
```

```

        % all elements are primes, so break
    end
end
pf=sort(pf);
% sort numbers in increasing order
disp(pf)
% display the result

```

```

Error using input
Cannot call INPUT from EVALC.

```

```

Error in from_higher (line 2)
c=input('enter a natural nubmer: ');

```

function

```

function [ans]=prime_factor(c)
global squarenumbers
% the out put has two elements (a+b) and (a-b)
a=floor(1/2*(c/3+3));
% take the upper boundary for a
ans=[];
% make the output as a vactor
while a^2-c>=0
    % stop after it reached the lower boundary
    B=a^2-c;
    % let B=b^2
    existsb=any(squarenumbers==B);
    % find from the square list
    if existsb==1
        b=find(squarenumbers==B)-1;
        % if B is a square, the column number-1 would be b since we started
        % from 0
        ans(end+1)=a+b;
        ans(end+1)=a-b;
        % add two factors into the list
        break
        % end when we find factors
    end
    a=a-1;
    % if we didn't find factor, repeat this process until we find one
    % here we have a=a-1 since we started from higher boundary
end
end

```

Published with MATLAB® R2023b